

POC d'un projet d'automatisation avec Selenium, Cucumber, Junit, Java, POM

Et

L'approche BDD



Centre de Formation : Proservices Compagny Training

Préparé par : Chaima MOSBEHI & Ines MATMATI

Formateur : Zied HANNACHI

Table des matières

1. Introduction 3
2. Technologies Utilisées 3
3. Design Pattern Utilisé 3
4. Approche BDD (Behavior Driven Development) 3
5. Fonctionnalités de la Library Fonctionnelle 4
6. Architecture du Projet 4
7. Exécution des Tests 4
8. Conclusion ..... **Erreur ! Signet non défini.**

1. Introduction

Ce document décrit la Proof of Concept (POC) pour l'automatisation des tests en utilisant Selenium, Cucumber, JUnit et Java. L'objectif est d'évaluer la faisabilité et l'efficacité de cette approche dans un contexte de test automatisé.

2. Technologies Utilisées

Selenium	Framework d'automatisation des tests pour les applications web	
Cucumber	Outil permettant l'écriture de tests en langage naturel (BDD - Behavior Driven Development).	
JUnit	Framework de test unitaire pour Java	
Java	Langage de programmation principal	

3. Design Pattern Utilisé

- **Page Object Model (POM)**

Le Page Object Model (POM) est un modèle de conception qui facilite la maintenance et la réutilisation du code en séparant la logique métier de la structure de la page web.

- ✓ **Avantages :**
  - Amélioration de la lisibilité et de la modularité du code.
  - Réduction de la redondance et de la maintenance.
  - Séparation entre la logique de test et l'interaction avec l'UI.

4. Approche BDD (Behavior Driven Development)

L'approche BDD permet d'écrire des scénarios de test compréhensibles par les parties prenantes (testeurs, développeurs, business).

Cucumber utilise le langage Gherkin pour décrire ces scénarios sous forme de **Given-When-Then**.

Exemple de Scénario BDD en Gherkin

```
Feature: Connexion utilisateur
  Scenario: Connexion avec des identifiants valides
    Given L'utilisateur est sur la page de connexion
    When Il saisit son nom d'utilisateur et son mot de passe valides
    And Il clique sur le bouton de connexion
    Then Il est redirigé vers la page d'accueil
```

5. Fonctionnalités de la Library Fonctionnelle

Une bibliothèque de fonctions réutilisables est mise en place pour standardiser les interactions avec l'UI et optimiser l'écriture des tests.

- **Gestion des éléments UI :** Localisation et interaction avec les éléments HTML (click, sendKeys, getText, etc.).
- **Gestion des assertions :** Vérifications automatisées des résultats attendus.
- **Gestion des événements :** Attente explicite, gestion des pop-ups, etc.
- **Gestion des rapports :** Génération de rapports détaillés pour le suivi des tests.

6. Architecture du Projet

Le projet suit une structure modulaire pour une meilleure organisation :

```
/src
  /main/java
    /pages    --> Classes du Page Object Model
    /utils    --> Fonctions utilitaires et gestion des logs
  /test/java
    /stepDefinitions    --> Implémentation des étapes BDD
    /runners    --> Configuration des exécutions Cucumber
  /resources
    /features    --> Fichiers Gherkin décrivant les scénarios
```

7. Exécution des Tests

Les tests peuvent être exécutés via différentes méthodes :

- Directement depuis l'IDE (IntelliJ, Eclipse)
- Via Maven : `mvn test`
- Intégration avec Jenkins pour exécution en CI/CD